

# Linear Regression and Regularization — Practice Problems

Instructor: Dr. Akash Gupta

## Table of contents

|                                  |    |
|----------------------------------|----|
| Part 1: Questions                | 2  |
| Part 2: Answers and Explanations | 24 |

## Part 1: Questions

### Question 1

A Zillow analyst trains a linear-regression model with one predictor: `square_footage` predicting `price`. What does the fitted line literally represent?

- a) The line that minimizes the SUM of squared vertical distances between the line and the training points (the line of best fit under OLS)
- b) The line that passes through every training point exactly
- c) The line with the smallest slope possible
- d) The line passing through the mean of  $x$  and the mean of  $y$  at a 45-degree angle
- e) The line that maximizes the predicted price

[☐ Jump to solution](#)

### Question 2

What does **Ordinary Least Squares (OLS)** minimize when fitting a linear regression?

- a) The maximum residual
- b) The sum of absolute residuals
- c) The number of features used
- d) The sum of SQUARED residuals between actual and predicted values
- e) The training accuracy

[☐ Jump to solution](#)

### Question 3

Why does OLS minimize the SQUARED residuals rather than the absolute residuals?

- a) Squared residuals are required by scikit-learn
- b) Absolute residuals always equal squared residuals
- c) Absolute values cannot be computed in numpy
- d) Squaring is faster on computers
- e) Squaring makes a model that is differentiable, has a clean closed-form solution, and penalizes large errors disproportionately — features that yield convenient math and useful behavior

[☐ Jump to solution](#)

#### Question 4

A Zillow simple linear model is  $\text{price} = 50,000 + 200 \times \text{sq\_ft}$ . What does the **slope of 200** mean in plain English?

- a) 200 square feet predicts a price of \$50,000
- b) Each additional square foot is associated with about \$200 more in predicted price
- c) The model is 200% accurate
- d) Houses larger than 200 sq ft are worth \$50,000 each
- e) The intercept is \$200

[□ Jump to solution](#)

#### Question 5

In an MLR model, what does the **intercept** ( $\beta_0$ ) represent?

- a) The largest predicted value
- b) The predicted value of the outcome when every predictor equals zero (sometimes meaningful, sometimes not — e.g., a “house with zero square feet” is not a real situation)
- c) The  $R^2$  of the model
- d) The smallest residual
- e) The variance of the target

[□ Jump to solution](#)

#### Question 6

A real-estate MLR model is:  $\text{price} = 50,000 + 200 \times \text{sq\_ft} + 15,000 \times \text{bathrooms} - 500 \times \text{age}$ . What does the coefficient **+15,000** on bathrooms mean?

- a) Holding  $\text{sq\_ft}$  and  $\text{age}$  constant, each additional bathroom is associated with about \$15,000 more in predicted price
- b) The intercept is \$15,000 too high
- c) Bathrooms account for 15,000% of the model's  $R^2$
- d) Houses with bathrooms are worth \$15,000
- e) Bathrooms are perfectly correlated with  $\text{sq\_ft}$

[□ Jump to solution](#)

### Question 7

A Zillow MLR has coefficient on age = -500. The minus sign tells you what?

- a) Older houses are predicted to be cheaper, on average — each additional year of age is associated with about \$500 less in predicted price, holding the other features constant
- b) The age column is incorrectly encoded
- c) Older houses cause lower prices
- d) The model is broken
- e) Age has no predictive power

[□ Jump to solution](#)

### Question 8

A LinkedIn salary model has unstandardized coefficients: years\_experience = +1,200, is\_phd = +12,000. Which statement about comparing the magnitudes is MOST accurate?

- a) The PhD effect is exactly 10 times stronger than years of experience
- b) PhD is more important than experience
- c) Experience is more important than PhD
- d) The two coefficients are not directly comparable because the units differ — years\_experience is measured in years, is\_phd is a 0/1 indicator. To compare relative impact, scale all features (e.g., StandardScaler) and re-examine the standardized coefficients
- e) They are directly comparable because they are both numeric

[□ Jump to solution](#)

### Question 9

What is the purpose of **standardized coefficients** (computing them after scaling features)?

- a) They allow the magnitudes of coefficients across DIFFERENT features to be compared on a common scale, since each coefficient now represents the change in y for a one-standard-deviation change in that feature
- b) They make the model train faster
- c) They eliminate multicollinearity
- d) They guarantee a higher  $R^2$
- e) They are required by scikit-learn

[□ Jump to solution](#)

**Question 10**

A categorical predictor `region`  $\in \{\text{West, East, South}\}$  is one-hot encoded as `region_East` and `region_South` (`region_West` dropped as the reference). What does the coefficient on `region_East` mean?

- a) The mean target value for East customers
- b) Nothing — categorical coefficients are uninterpretable
- c) The percent of customers in the East
- d) The slope between East and South
- e) The expected difference in  $y$  between East and the REFERENCE category (West), holding the other predictors constant — interpret it as “East customers, on average, have an outcome that is `coef_East` higher (or lower) than otherwise-identical West customers”

[☐ Jump to solution](#)

**Question 11**

A regression on the test set reports  $SS_{\text{res}} = 1,500$  and  $SS_{\text{tot}} = 10,000$ . What is the test  $R^2$ ?

- a) 0.10
- b) 0.25
- c) 0.50
- d) 0.95
- e) 0.85

[☐ Jump to solution](#)

**Question 12**

Test  $R^2 = 0.85$  means:

- a) The model is correct 85% of the time on classification
- b) 85% of features were used
- c) 85% confidence that the model is correct
- d) About 85% of the variation in the test-set target is captured by the model — strong fit
- e) The model's threshold is 0.85

[☐ Jump to solution](#)

## Question 13

When comparing two MLR models with **different numbers of predictors** on the same data, why prefer **adjusted  $R^2$**  over plain  $R^2$ ?

- a) Adjusted  $R^2$  is always higher
- b) Plain  $R^2$  never decreases when you add a feature, so it can be inflated by piling on irrelevant predictors. Adjusted  $R^2$  imposes a penalty proportional to the number of predictors so it stays honest as you compare models
- c) Adjusted  $R^2$  is faster to compute
- d) Adjusted  $R^2$  is the only one supported by scikit-learn
- e) They are mathematically identical

[□ Jump to solution](#)

## Question 14

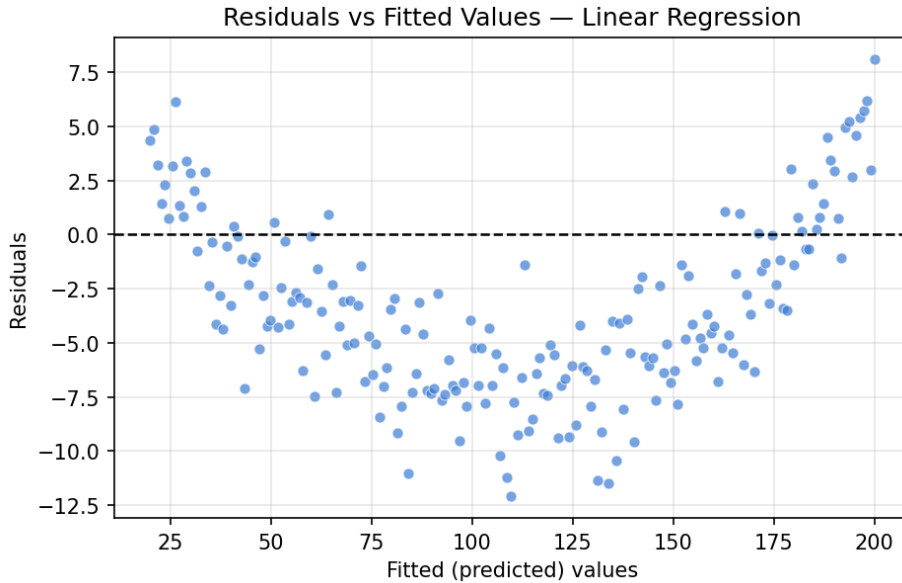
True or false:  $R^2$  on a held-out test set is always between 0 and 1.

- a) True — it is a probability
- b) True — sklearn enforces it
- c) False —  $R^2$  is always above 1
- d) True —  $R^2$  is bounded by definition
- e) False —  $R^2$  can be NEGATIVE on the test set when the model performs worse than predicting the mean of  $y$  for every row, because the formula  $R^2 = 1 - SS_{\text{res}}/SS_{\text{tot}}$  is unbounded below

[□ Jump to solution](#)

## Question 15

A Zillow analyst plots residuals against fitted values for a linear-regression home-price model:



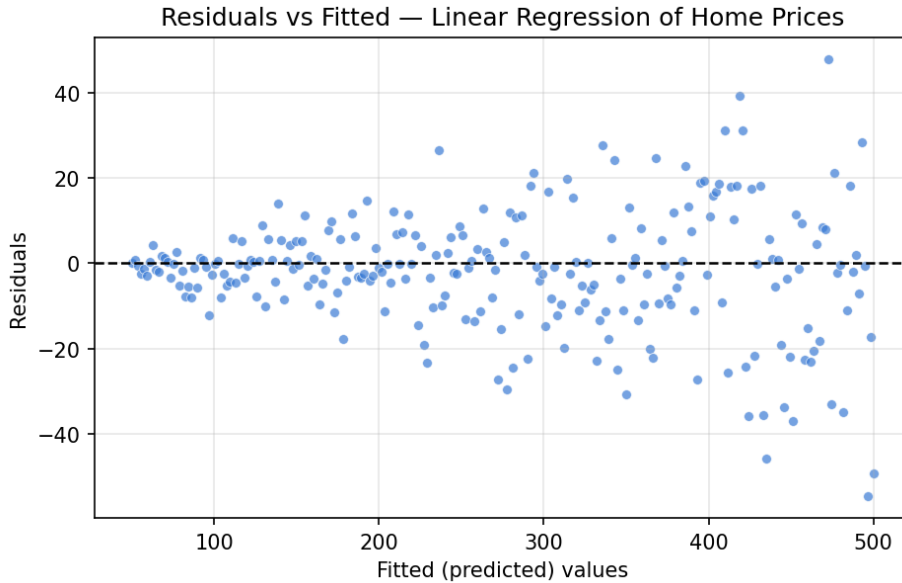
What does this plot indicate?

- a) Multicollinearity
- b) The dataset is too small
- c) The model is overfitting
- d) The residuals are normally distributed — no action needed
- e) The linearity assumption is violated — the true relationship is curved. Add a polynomial / log term, transform a predictor, or use a non-linear model

☐ [Jump to solution](#)

### Question 16

A different Zillow regression produces this residuals-vs-fitted plot — the residual SPREAD grows with the fitted values:



Which assumption is violated?

- a) Linearity
- b) Normality
- c) Independence
- d) Regularization
- e) Homoscedasticity (constant residual variance) — fixes include log-transforming the target, weighted regression, or moving to a model that does not assume constant variance

[□ Jump to solution](#)

### Question 17

A team trains a linear regression on **monthly retail sales over 5 years** without time-aware splitting. Which standard assumption is violated by treating these rows as i.i.d.?

- a) Linearity
- b) Homoscedasticity
- c) Normality of residuals
- d) Independence — successive months are correlated; ignoring this yields underestimated standard errors and a model that may look great in cross-validation but fail at forecasting truly future months
- e) Multicollinearity

[□ Jump to solution](#)



**Question 18**

A single Zillow training row has `square_footage = 50,000` (an enormous outlier — typo for 5,000). On a fitted OLS model, this single row can:

- a) Have no effect on the line
- b) Disproportionately pull the OLS line toward itself, distorting all coefficients — OLS minimizes squared residuals, so a single huge residual dominates the loss. Inspect, correct, or use a robust regression
- c) Improve model accuracy
- d) Improve  $R^2$  guaranteed
- e) Be ignored automatically by scikit-learn

[☐ \*Jump to solution\*](#)

**Question 19**

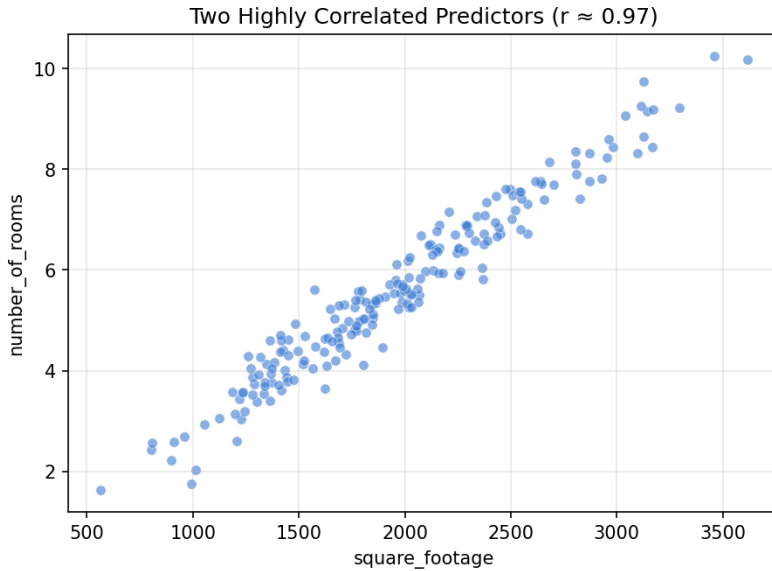
What is **multicollinearity**?

- a) A model trained on multiple computers
- b) Two or more predictors that are highly correlated with EACH OTHER (not with the target), causing the model to have trouble distinguishing their individual effects
- c) Outliers in the target
- d) A type of cross-validation
- e) Multiple targets being predicted at once

[☐ \*Jump to solution\*](#)

**Question 20**

A Redfin model has `square_footage` coefficient of \$+200. When `number_of_rooms` is added, the coefficient on `square_footage` flips to \$-50. The scatter of the two predictors is shown below.



What is happening?

- a) A bug in scikit-learn
- b) `number_of_rooms` is a leakage feature
- c) Overfitting
- d) The intercept is wrong
- e) Multicollinearity — `square_footage` and `number_of_rooms` are highly correlated, so the two coefficients “fight” for the same explanatory power and individual coefficients become unstable. Prediction may still be fine, but each coefficient should not be trusted individually

[□ Jump to solution](#)

### Question 21

What does a **Variance Inflation Factor (VIF)** above 10 typically indicate?

- a) The feature is useless and must be dropped
- b) The feature is perfectly uncorrelated with other features
- c) The model has reached convergence
- d) Serious multicollinearity — that predictor is highly explained by the OTHER predictors. Common rules of thumb:  $VIF < 5$  fine,  $5-10$  watch,  $> 10$  act on it (drop, combine, or use Ridge)
- e) The feature is the most important predictor

[□ Jump to solution](#)

### Question 22

Multicollinearity hurts the model's INTERPRETATION more than its PREDICTION. Why?

- a) The coefficients become unstable (sensitive to small data changes), so individual coefficient interpretations are unreliable; the model's combined predictions can still be reasonable because the correlated predictors cover for each other's wobble
- b) Multicollinearity always destroys predictions
- c) Predictions are unaffected because OLS does not use coefficients
- d) Interpretation is impossible for any regression
- e) Multicollinearity has no effect at all

[☐ Jump to solution](#)

### Question 23

Which of the following is NOT a standard remedy for multicollinearity?

- a) Drop one of the correlated predictors
- b) Combine the correlated predictors into one (e.g., a ratio or PCA component)
- c) Add MORE highly correlated predictors so the model has more options
- d) Switch to a tree-based model that is less sensitive to it
- e) Use Ridge regression, which spreads the coefficient mass across correlated features

[☐ Jump to solution](#)

### Question 24

In a Lending Club credit-risk model with 200 raw features (many derived from a few originals), what is the BIGGEST danger when fitting plain OLS?

- a) The model trains too slowly
- b) The  $R^2$  is exactly 1.0 by definition
- c) The model overfits — many features mean many parameters; OLS will fit training-data noise (extreme coefficients in opposing directions) and generalize poorly. Regularization (Ridge, Lasso, Elastic Net) is the standard remedy
- d) Plain OLS cannot handle 200 features
- e) The model is automatically biased toward zero

[☐ Jump to solution](#)

### Question 25

How does adding **regularization** to a linear model affect the bias-variance tradeoff?

- a) Regularization removes both bias and variance simultaneously
- b) Regularization eliminates bias and increases variance
- c) Regularization affects neither bias nor variance
- d) Regularization increases bias slightly and reduces variance substantially — usually a favorable tradeoff for noisy or high-dimensional data
- e) Regularization always increases both bias and variance

[□ Jump to solution](#)

### Question 26

Mathematically, what does **Ridge regression** add to the OLS objective?

- a) Nothing — Ridge is identical to OLS
- b) A penalty proportional to  $R^2$
- c) A penalty proportional to the SUM OF ABSOLUTE coefficients (L1 penalty)
- d) A penalty proportional to the number of features
- e) A penalty proportional to the SUM OF SQUARED coefficients (L2 penalty), which shrinks every coefficient toward zero proportionally

[□ Jump to solution](#)

### Question 27

What is the typical EFFECT of Ridge on coefficients?

- a) Shrinks all coefficients toward zero — they get smaller in magnitude but rarely reach exactly zero. Useful when you believe MOST features carry some signal and you just want to control the size
- b) Leaves all coefficients unchanged from OLS
- c) Sets many coefficients to zero
- d) Makes coefficients larger to compensate
- e) Makes the intercept zero

[□ Jump to solution](#)

### Question 28

Mathematically, what does **Lasso regression** add to the OLS objective?

- a) A penalty proportional to the SUM OF ABSOLUTE coefficients (L1) — the geometry of the L1 ball causes the optimum to land on coordinate axes, which is why Lasso can drive coefficients exactly to zero
- b) A penalty proportional to the sum of squared coefficients (L2)
- c) A polynomial penalty
- d) A penalty on the intercept
- e) Nothing — Lasso is identical to Ridge

☐ [Jump to solution](#)

### Question 29

What is the typical EFFECT of Lasso on coefficients?

- a) Shrinks all of them by the same factor
- b) Drives some coefficients exactly to zero, effectively dropping those features from the model — Lasso doubles as automatic feature selection
- c) Leaves coefficients unchanged
- d) Always sets every coefficient to zero
- e) Makes the intercept zero

☐ [Jump to solution](#)

### Question 30

A Capital One team has **200 features** in a credit-risk model and suspects only **about 30 of them really matter**. Which regularizer is most aligned with that prior belief?

- a) Ridge — it shrinks all coefficients but never to zero
- b) Polynomial features
- c) OLS — no regularization needed
- d) Lasso — it drives the coefficients of irrelevant features to exactly zero, producing a sparse, interpretable model that uses only the ~30 features that survive
- e) Standardization

☐ [Jump to solution](#)

### Question 31

When would you PREFER Ridge over Lasso?

- a) Always — Ridge is mathematically superior
- b) When you believe MOST of the features carry some signal (so dropping any of them outright is wasteful) and you want stable, well-conditioned shrinkage — particularly with correlated features that Lasso would arbitrarily pick one of
- c) When you have only a single feature
- d) When the target is binary
- e) Never — Lasso always wins

☐ [Jump to solution](#)

### Question 32

What is **Elastic Net** and when is it most useful?

- a) A neural network architecture
- b) A combination of Ridge (L2) and Lasso (L1) penalties. Useful when you have GROUPS of correlated features and want feature selection (Lasso would arbitrarily pick one of each correlated group; Elastic Net can keep multiple)
- c) A type of cross-validation
- d) Only for classification
- e) A way to scale features

☐ [Jump to solution](#)

### Question 33

Why is it important to **scale (standardize) features BEFORE applying Ridge or Lasso**?

- a) sklearn refuses to fit otherwise
- b) The penalty acts on coefficient MAGNITUDE. Without scaling, a feature in the millions (income) is naturally given a tiny coefficient and a feature in  $[0, 1]$  (a percentage) is given a huge coefficient — purely because of unit choice. The regularization penalty then lands unevenly across features. Scaling puts every feature on the same footing so the penalty applies fairly
- c) Scaling is only required for tree models
- d) Without scaling,  $R^2$  is undefined
- e) Scaling is purely cosmetic

☐ [Jump to solution](#)

### Question 34

What does the **alpha** hyperparameter control in `Ridge(alpha=...)` and `Lasso(alpha=...)`?

- a) The number of features
- b) The size of the test set
- c) The strength of the regularization penalty:  $\alpha = 0$  reduces to plain OLS; very large  $\alpha$  shrinks all coefficients toward zero (Ridge) or kills most of them (Lasso). It is tuned via cross-validation
- d) The intercept
- e) The  $R^2$  of the model

[☐ Jump to solution](#)

### Question 35

What does `Ridge(alpha=0)` reduce to?

- a) Lasso
- b) Constant prediction
- c) An error
- d) Random predictions
- e) Plain OLS — with zero penalty, the regularized objective collapses to the OLS objective

[☐ Jump to solution](#)

### Question 36

What happens to the coefficients as  $\alpha \rightarrow \infty$ ?

- a) They explode to infinity
- b) They settle at the OLS values
- c) They all approach ZERO (Ridge) or are exactly zero (Lasso) — the model becomes a constant predictor (the intercept)
- d) The intercept goes to infinity
- e) The  $R^2$  climbs toward 1

[☐ Jump to solution](#)

### Question 37

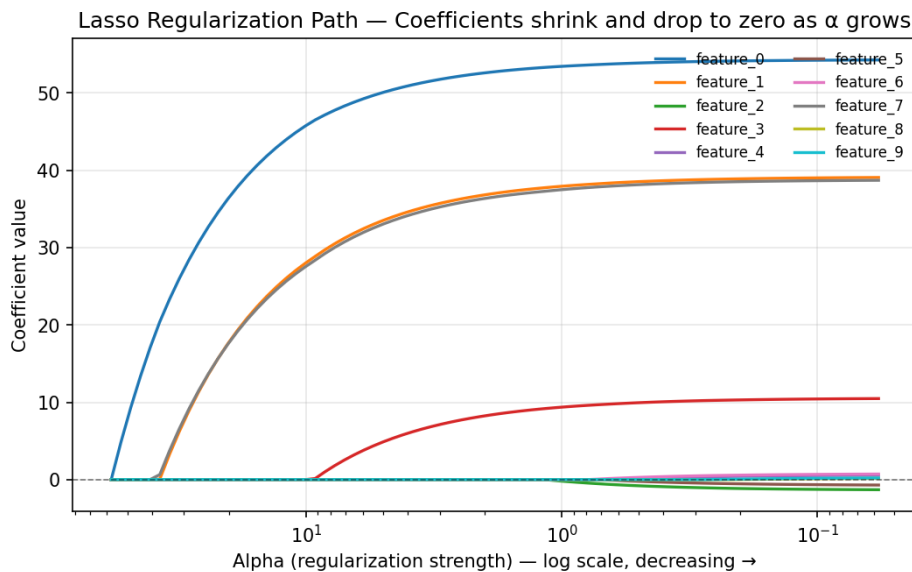
How is alpha typically CHOSEN in practice?

- a) Set it to 1 because that is sklearn's default
- b) Pick the alpha that maximizes the training  $R^2$
- c) Pick the largest alpha allowed
- d) Cross-validation: try a grid of alpha values, pick the one that gives the best out-of-fold metric (use RidgeCV / LassoCV / ElasticNetCV, or GridSearchCV over a Pipeline)
- e) Use 0.05

[□ Jump to solution](#)

### Question 38

A team plots Lasso coefficient values along a grid of alpha (the **regularization path**) for a 10-feature dataset:



Which statement BEST describes what the plot shows?

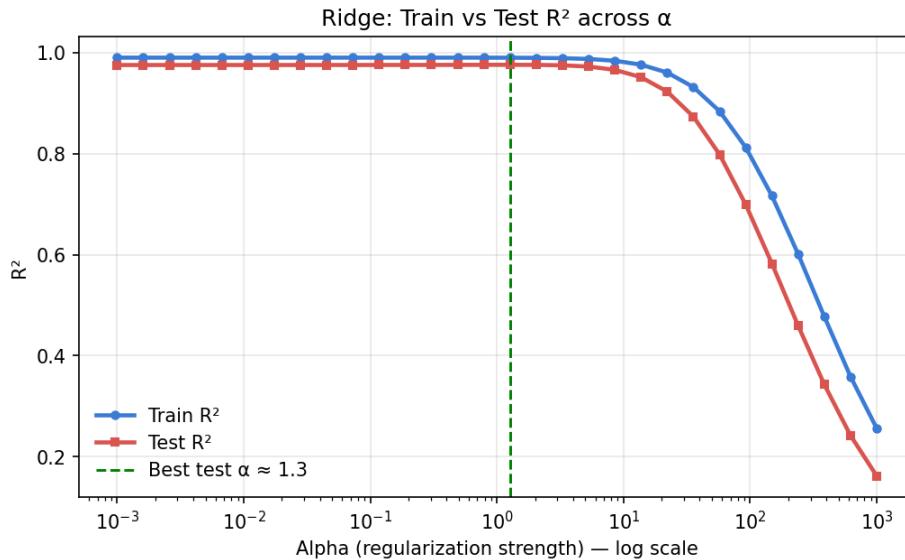
- a) All coefficients stay constant
- b) Coefficients grow as alpha grows
- c) All coefficients become negative
- d) As alpha increases, coefficients shrink — and one by one, individual coefficients hit exactly zero (the “feature drops out” event). Inspecting the path tells you which features survive at each level of regularization
- e) The intercept becomes infinite

[□ Jump to solution](#)



### Question 39

A team trains a Ridge regression at many values of alpha (regularization strength) on a 50-feature dataset and plots train and test  $R^2$  for each:



Which BEST describes what the plot is telling them?

- a) The smallest alpha is best because train  $R^2$  is highest there
- b) The largest alpha is best because the regularization is strongest there
- c) Train  $R^2$  stays near 1.0 across all alphas, so any alpha is fine
- d) Test  $R^2$  peaks at a moderate alpha (the sweet spot near the dashed line); below it the model overfits, above it the model is over-regularized. Pick the alpha where test  $R^2$  peaks
- e) The plot proves Ridge cannot be tuned

[Jump to solution](#)

### Question 40

A teammate computed VIF on every feature and one feature scored  $VIF = 87$ . Which action is MOST appropriate?

- a) Ignore it
- b) Add more correlated features
- c) Investigate that feature: it is almost a linear combination of the others. Drop it, combine it with the related features, or move to Ridge regression which is far less sensitive to multicollinearity
- d) Switch to Lasso, which always solves multicollinearity by random selection
- e) Increase the test set size

[Jump to solution](#)

**Question 41**

Consider:

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
print(model.intercept_)
print(model.coef_)
print(model.score(X_test, y_test))
```

What does each of the three print lines show?

- a) The number of rows; the number of features; the loss
- b) The training  $R^2$ ; nothing meaningful; the training  $R^2$
- c) The intercept (a single scalar); the coefficient array (one entry per predictor column in  $X_{\text{train}}$ ); the test-set  $R^2$
- d) The training MSE; the test MSE; the training MAE
- e) Three identical numbers

[□ Jump to solution](#)

**Question 42**

Consider:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge

pipe = Pipeline([
    ("scale", StandardScaler()),
    ("model", Ridge(alpha=1.0)),
])
pipe.fit(X_train, y_train)
```

Why is wrapping StandardScaler and Ridge in a Pipeline the right pattern?

- a) Pipelines run faster
- b) Pipelines reduce memory usage
- c) The pipeline guarantees the scaler is fit on the training fold ONLY (no leakage during cross-validation), and the same fitted scaler is then applied to test data automatically. Regularized models in particular need scaling, and the pipeline ensures the two steps are always coupled correctly
- d) Pipelines avoid the need for train\_test\_split
- e) Required by scikit-learn

[□ Jump to solution](#)

**Question 43**

Consider:

```
from sklearn.linear_model import Lasso
import numpy as np

lasso = Lasso(alpha=0.1).fit(X_train_scaled, y_train)
kept = np.array(feature_names)[lasso.coef_ != 0]
print(f"Lasso kept {len(kept)} of {X_train_scaled.shape[1]} features:")
print(kept)
```

What is this snippet doing?

- a) Computing  $R^2$
- b) Printing all features used in the model
- c) Identifying which features Lasso retained — the features whose coefficients did not get driven to zero by the L1 penalty. This is Lasso's built-in feature-selection output
- d) Performing cross-validation
- e) Predicting on test data

[□ Jump to solution](#)

**Question 44**

Consider:

```
from sklearn.linear_model import RidgeCV
ridge_cv = RidgeCV(alphas=[0.01, 0.1, 1, 10, 100], cv=5).fit(X_train, y_train)
print(ridge_cv.alpha_)
```

What does `ridge_cv.alpha_` return?

- a) The alpha value from the supplied grid that achieved the best 5-fold cross-validation score; in production, you would refit Ridge with this alpha (or just use `ridge_cv` directly)
- b) The number of features
- c) The mean test  $R^2$
- d) The training set size
- e) Always 1.0

[□ Jump to solution](#)

**Question 45**

Consider:

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X_train, y_train,
                          cv=5, scoring="neg_mean_squared_error")
rmse = (-scores) ** 0.5
print(rmse.mean(), rmse.std())
```

Why is the scoring "neg\_mean\_squared\_error" and why the negation?

- a) Bug — should be "mean\_squared\_error"
- b) RMSE is unsupported
- c) sklearn does not support MSE
- d) The minus sign is a typo
- e) Scikit-learn's convention is that "higher is better" for `cross_val_score`; since lower MSE is better, sklearn returns the NEGATIVE of MSE. Flipping the sign and taking the square root recovers RMSE for a more interpretable summary

[□ Jump to solution](#)

#### Question 46

A linear model trained on a Lending Club dataset has **train  $R^2 = 0.97$**  and **test  $R^2 = 0.51$** . Which action is MOST likely to help?

- a) Add MORE features
- b) Increase the polynomial degree
- c) Drop the test set entirely
- d) The model is overfitting. Add regularization (Ridge, Lasso, or Elastic Net via cross-validated alpha), reduce the feature count (drop weak / collinear features), gather more training data, or simplify the model
- e) Report only the train  $R^2$

[□ Jump to solution](#)

#### Question 47

A team has a credit dataset with **a few groups of highly correlated features** (e.g., several income-related columns) and wants the model to KEEP useful features from each group rather than arbitrarily picking one. Which regularizer matches this need BEST?

- a) Plain OLS
- b) Elastic Net — combines L1 (selection) and L2 (stable handling of correlated groups), often retaining MULTIPLE features from each correlated cluster
- c) Ridge alone — it would not perform feature selection
- d) Lasso alone — it would arbitrarily zero out all but one in each correlated group
- e) Switch to a tree model

[□ Jump to solution](#)

## Question 48

A team applies **Lasso** to a credit dataset with severe multicollinearity. They notice the surviving features change wildly between random splits of the data. Which best explains this?

- a) Lasso is broken
- b) Random sampling is broken
- c) The team forgot to scale
- d) When several features are correlated, Lasso's solution is not unique — it tends to pick ONE of them somewhat arbitrarily, and the "winner" can change with small perturbations of the training data. Ridge or Elastic Net produces more stable behavior
- e) The dataset is too small

[☐ Jump to solution](#)

## Question 49

A real-estate analyst transforms `price` to `log(price)` and refits a linear model. The residual plot is now well-behaved. What is the MOST useful side effect of the log transform?

- a) The model is no longer a linear regression
- b) Coefficients now read directly as percentages
- c)  $R^2$  is guaranteed to reach 1.0
- d) The intercept is now zero
- e) Multiplicative effects (a 10% bigger house earns roughly the same percentage premium across the price range) become additive in log-space, often producing more linear relationships and stabilizing residual variance

[☐ Jump to solution](#)

## Question 50

Adjusted  $R^2$  for a model with  **$R^2 = 0.85$ ,  $n = 100$  observations,  $p = 10$  predictors** is approximately:

- a) 0.83
- b) 0.85
- c) 1.00
- d) 0.50
- e) 0.15

[☐ Jump to solution](#)

**Question 51**

A bank is asked by a regulator to EXPLAIN the model's decision for a specific loan applicant. Which of the following models is hardest to defend on individual-level interpretability?

- a) Plain linear / logistic regression with named features
- b) Ridge regression on standardized features
- c) A 500-tree Random Forest with hundreds of complex interactions and non-linear splits
- d) Lasso regression with a sparse, named subset of features
- e) A single shallow decision tree

☐ [Jump to solution](#)

**Question 52**

A scenario: you have 500 features, only ~50 truly matter, and the rest are noise. You want both prediction quality and a small, interpretable feature set. Which is the BEST starting recipe?

- a) Plain OLS on all 500 features
- b) Drop 450 features by hand at random
- c) StandardScaler + Lasso (or Elastic Net), with alpha chosen by cross-validation; inspect the surviving features and report them
- d) PolynomialFeatures(degree=3) followed by OLS
- e) Random Forest

☐ [Jump to solution](#)

**Question 53**

A scenario: a team's OLS model has unstable coefficients (sign flips when a single row is added or removed) but acceptable test predictions. The likely cause and best first step?

- a) Underfitting — switch to a deeper model
- b) Multicollinearity. Compute VIF, drop or combine the most-correlated feature group, and / or switch to Ridge regression (which spreads coefficient mass across correlated features and gives stable, well-conditioned solutions)
- c) The data has no signal
- d) Random\_state is wrong
- e) Add more features

☐ [Jump to solution](#)

**Question 54**

A classic OLS interpretation pitfall: the team finds that ice-cream sales correlate strongly with sunburns and concludes “ice-cream sales cause sunburns.” Which assumption did they confuse?

- a) Linearity
- b) Homoscedasticity
- c) Correlation vs. CAUSATION — both are driven by warm weather (a confounder). Linear regression reports associations, not causal effects, unless the underlying study design (random assignment) supports a causal claim
- d) Independence
- e) Normality

[□ Jump to solution](#)

**Question 55**

A Lending Club credit-pricing team trains an Elastic-Net model with cross-validated alpha and reports **train  $R^2 = 0.62$ , test  $R^2 = 0.60$** . They are puzzled — neither number is impressive. Which is the BEST response?

- a) The numbers are CLOSE — the model generalizes well and is not overfitting.  $R^2 = 0.60$  means about 60% of the variance is explained, which is realistic for noisy real-world credit data. Either accept the result, gather better features, or move to a more flexible model only if the marginal gain justifies the loss of interpretability
- b) Switch to Random Forest immediately
- c) The model is overfitting — increase the alpha further
- d) Drop regularization
- e) The  $R^2$  of 0.60 indicates the data is corrupt

[□ Jump to solution](#)

## Part 2: Answers and Explanations

### Answer 1

**Correct: (a).** Linear regression's fitted line is the one that minimizes the SUM of squared vertical distances (residuals) between the line and the training points. It does not pass through every point; it balances them. This is what "of best fit under OLS" means.

[□ Back to question](#)

### Answer 2

**Correct: (d).** OLS minimizes  $\sum (y_i - \hat{y}_i)^2$  — the sum of squared residuals. This objective has a clean closed-form solution and gives the model many of its statistical properties (unbiasedness under standard assumptions).

[□ Back to question](#)

### Answer 3

**Correct: (e).** Squaring is mathematically convenient (differentiable, closed-form solution) and behaviorally useful (large errors are penalized disproportionately, so the line cannot ignore severe misses).

[□ Back to question](#)

### Answer 4

**Correct: (b).** A coefficient of 200 means each additional unit of the predictor is associated with about 200 units of the predicted outcome. Here, each additional square foot adds about \$200 to predicted price.

[□ Back to question](#)

### Answer 5

**Correct: (b).** The intercept is the predicted outcome when every predictor equals zero. Sometimes that is meaningful (baseline customer); sometimes it is not (no real house has 0 sq ft). Either way, it anchors the line — do not over-interpret it on its own.

[□ Back to question](#)



## Answer 6

**Correct: (a).** The “all else equal” qualifier is the heart of MLR coefficient interpretation. Each additional bathroom is associated with about \$15,000 more in predicted price, holding `sq_ft` and `age` fixed. Without that qualifier the interpretation is wrong.

[□ Back to question](#)

## Answer 7

**Correct: (a).** A negative coefficient means the predictor moves the prediction in the opposite direction. Each additional year of age is associated with about \$500 less in predicted price, holding the other features fixed. Note: “associated with” — not “causes.”

[□ Back to question](#)

## Answer 8

**Correct: (d).** Coefficient magnitudes are NOT directly comparable across features measured in different units. Years of experience ranges 0–40+; the PhD indicator is 0/1. To compare relative effect, standardize the features (e.g., `StandardScaler`) and look at the resulting coefficients on a common per-standard-deviation scale.

[□ Back to question](#)

## Answer 9

**Correct: (a).** Standardized coefficients are computed from features that have been centered to mean 0 and scaled to standard deviation 1. Each coefficient now represents the change in  $y$  for a one-standard-deviation change in that feature, which makes magnitudes comparable across features measured in different units.

[□ Back to question](#)

## Answer 10

**Correct: (e).** When one category is dropped (the “reference”), each remaining dummy’s coefficient is the expected difference in  $y$  between that category and the reference, holding other predictors constant. East customers have an outcome on average `coef_East` higher (or lower) than otherwise-identical West customers.

[□ Back to question](#)

## Answer 11

**Correct: (e) 0.85.**  $R^2 = 1 - SS_{\text{res}}/SS_{\text{tot}} = 1 - 1,500 / 10,000 = 1 - 0.15 = 0.85$ .

[□ Back to question](#)

## Answer 12

**Correct: (d).**  $R^2$  is the proportion of variance in the test target captured by the model.  $R^2 = 0.85$  means about 85% of the variation is explained — strong fit on a real-world dataset.

[□ Back to question](#)

## Answer 13

**Correct: (b).** Plain  $R^2$  never decreases when you add a predictor, so a “kitchen-sink” model can always look better. Adjusted  $R^2$  imposes a per-predictor penalty, so it can decrease when you add useless features — making it the more honest metric for comparing models with different feature counts.

[□ Back to question](#)

## Answer 14

**Correct: (e).**  $R^2 = 1 - SS_{\text{res}}/SS_{\text{tot}}$  has no lower bound. On a held-out test set, a model that performs WORSE than predicting the mean of  $y$  on every row produces  $SS_{\text{res}} > SS_{\text{tot}}$ , so  $R^2 < 0$ . Negative test  $R^2$  is a clear signal something is wrong (bug, leakage, or extreme distribution shift).

[□ Back to question](#)

## Answer 15

**Correct: (e).** A U-shape in residuals vs fitted means the linear model systematically over-predicts in the middle and under-predicts at the extremes (or vice versa) — the true relationship is curved. Add a polynomial term, log-transform a predictor, or move to a non-linear model.

[□ Back to question](#)

## Answer 16

**Correct: (e) Homoscedasticity.** OLS assumes constant residual variance across all fitted values. A fan shape ("residuals spread grows with predictions") is heteroscedasticity — prediction intervals widen at high values. Common fixes: log-transform the target, weighted regression, or a model that does not assume constant variance.

[□ Back to question](#)

## Answer 17

**Correct: (d) Independence.** Successive months of retail sales are autocorrelated (last month's sales predict this month's). Standard OLS assumes residuals are independent; ignoring autocorrelation gives optimistic standard errors and CV scores. Use time-series CV (`TimeSeriesSplit`), include lagged features, or use a time-series model.

[□ Back to question](#)

## Answer 18

**Correct: (b).** OLS minimizes squared residuals, so a single huge mistake dominates the loss and pulls the line toward the outlier. Inspect the row, correct typos, consider robust regression (Huber, RANSAC), or remove genuine bad-data points before fitting.

[□ Back to question](#)

## Answer 19

**Correct: (b).** Multicollinearity is high correlation BETWEEN predictors. The model can still fit a useful prediction function but cannot cleanly attribute effect to one collinear feature vs. another — coefficients become unstable and interpretation suffers.

[□ Back to question](#)

## Answer 20

**Correct: (e).** Multicollinearity. Two correlated predictors fight for the same explanatory power; small data changes can push the explanation onto one feature or the other, flipping signs. Predictions are usually still fine; individual coefficient interpretations are not trustworthy. Use VIF to detect and Ridge to mitigate.

[□ Back to question](#)

## Answer 21

**Correct: (d).** A VIF of, say, 10 means the standard error of that coefficient is about  $\sqrt{10} \approx 3.2$  times larger than it would be without multicollinearity. Common thresholds: < 5 fine, 5–10 watch, > 10 act on it (drop, combine, or move to Ridge).

[□ Back to question](#)

## Answer 22

**Correct: (a).** Multicollinearity inflates the variance of individual coefficient estimates. Predictions are typically robust because the correlated predictors collectively still capture the signal. Interpretation is what suffers — and is exactly the use case where regularization (Ridge / Elastic Net) helps stabilize.

[□ Back to question](#)

## Answer 23

**Correct: (c).** Adding MORE highly correlated predictors makes the problem worse, not better. Standard remedies: drop one, combine into a ratio or PCA component, switch to Ridge, or use a tree-based model.

[□ Back to question](#)

## Answer 24

**Correct: (c).** With 200 features and a finite training set, OLS can fit training noise — extreme positive and negative coefficients on collinear features that perfectly cancel on training data but fail on new data. Regularization is the standard remedy.

[□ Back to question](#)

## Answer 25

**Correct: (d).** Regularization deliberately introduces a small bias (the penalty pulls coefficients away from the OLS values) to substantially reduce variance (less sensitive to noise in the training set). For most messy real-world datasets, the tradeoff is favorable on test performance.

[□ Back to question](#)

## Answer 26

**Correct: (e).** Ridge adds the L2 penalty  $\lambda \sum \beta_j^2$  to the OLS sum of squared residuals. The squared-coefficient penalty shrinks every coefficient proportionally toward zero but rarely (never, for a generic problem) sets them exactly to zero.

[□ Back to question](#)

## Answer 27

**Correct: (a).** Ridge shrinks every coefficient toward zero but does not zero them out. This is the right choice when you believe most features carry some signal and you want stable, well-conditioned shrinkage rather than aggressive feature pruning.

[□ Back to question](#)

## Answer 28

**Correct: (a).** Lasso adds the L1 penalty  $\lambda \sum |\beta_j|$ . The geometry of the L1 ball (corners on the coordinate axes) causes the constrained-optimum to land on those corners, which means individual coefficients become exactly zero — the source of Lasso's automatic feature selection.

[□ Back to question](#)

## Answer 29

**Correct: (b).** Lasso zeros out the coefficients of features it deems unhelpful, leaving a sparse model. The non-zero features are the ones the model actually uses — a built-in feature-selection mechanism.

[□ Back to question](#)

## Answer 30

**Correct: (d) Lasso.** With a strong prior that only a small subset of the 200 features carries signal, Lasso's L1 penalty drives the irrelevant coefficients to exactly zero, producing a sparse, interpretable model that uses only the surviving  $\sim 30$ . Ridge would shrink all 200 but keep them all in the model.

[□ Back to question](#)

## Answer 31

**Correct: (b).** Ridge is preferable when most features carry some signal, when you want stable shrinkage rather than abrupt elimination, and when the data has correlated feature groups (Lasso would pick one feature per group somewhat arbitrarily — Ridge spreads the coefficient mass across them).

[□ Back to question](#)

## Answer 32

**Correct: (b).** Elastic Net combines the L1 penalty (Lasso, for selection) and the L2 penalty (Ridge, for stability with correlated features). With grouped correlated features, Elastic Net often keeps multiple members of each group instead of arbitrarily picking one — usually the right behavior in real-world data.

[□ Back to question](#)

## Answer 33

**Correct: (b).** Regularization penalizes the magnitudes of the coefficients. Without scaling, the magnitudes reflect both feature importance and the units of measurement — so the penalty acts unevenly across features purely because of scale. Standardize features (e.g., `StandardScaler` inside a `Pipeline`) so the penalty applies fairly. Plain OLS does not have this issue and does not need scaling for accuracy.

[□ Back to question](#)

## Answer 34

**Correct: (c).** Alpha is the strength of the regularization penalty.  $\alpha = 0$  is plain OLS; very large alpha shrinks all coefficients toward zero (Ridge) or kills most of them (Lasso). The right alpha is found via cross-validation, not picked by hand.

[□ Back to question](#)

## Answer 35

**Correct: (e).** With  $\alpha = 0$ , the regularization term vanishes and the optimization reduces to the plain OLS objective. (In practice you would just call `LinearRegression`; setting  $\alpha=0$  in Ridge is fine but emits a numerical warning.)

[□ Back to question](#)

## Answer 36

**Correct: (c).** As alpha grows large the penalty dominates the loss and pushes coefficients toward zero. With Lasso, most coefficients hit exactly zero; with Ridge, all approach (but rarely reach) zero. The model collapses to predicting the intercept (the mean of  $y$  on the training set).

[□ Back to question](#)

## Answer 37

**Correct: (d).** Cross-validation. Define a grid of alpha values (often log-spaced), evaluate cross-validated performance at each, and pick the alpha with the best out-of-fold metric. RidgeCV, LassoCV, ElasticNetCV and GridSearchCV over a Pipeline are the standard tools.

[□ Back to question](#)

## Answer 38

**Correct: (d).** As alpha increases, Lasso's penalty pulls coefficients toward zero. Coefficients shrink smoothly, then hit exactly zero one by one. Plotting the path is a useful diagnostic: it shows which features survive the longest under increasing penalty (i.e., which the data "really wants").

[□ Back to question](#)

## Answer 39

**Correct: (d).** The classic regularization pattern: at very small alpha the model overfits (train  $R^2$  high, test  $R^2$  lower); at very large alpha the model is over-regularized (both fall as the model collapses toward predicting the mean); somewhere in between, test  $R^2$  peaks — that is the sweet spot. The dashed line marks the best alpha by test performance. Pick the alpha at the test- $R^2$  peak (in practice, via cross-validation).

[□ Back to question](#)

## Answer 40

**Correct: (c).**  $VIF = 87$  is a strong signal that the feature is almost a linear combination of the others. Investigate its meaning, drop it, combine it with the related features (e.g., a ratio or a PCA component), or switch to Ridge regression which is far less sensitive to multicollinearity than OLS.

[□ Back to question](#)

## Answer 41

**Correct: (c).** `model.intercept_` is the single scalar intercept; `model.coef_` is the array of coefficients, one per predictor column in `X_train`; `model.score(X_test, y_test)` returns the test-set  $R^2$ . The `(y_true, y_pred)` order does not matter for `.score` because it computes  $R^2$  on the model's own predictions on `X_test`.

[□ Back to question](#)

## Answer 42

**Correct: (c).** Inside a Pipeline, the scaler is refit on each training fold during cross-validation; no test rows ever influence the scaler — leakage-free by construction. The model is also automatically chained, so you cannot accidentally forget to scale at prediction time. This pattern is essential for any regularized linear model.

[□ Back to question](#)

## Answer 43

**Correct: (c).** Lasso drives coefficients of unhelpful features to exactly zero. Subsetting `feature_names` by `lasso.coef_ != 0` reveals the surviving subset — Lasso's built-in feature-selection output, which can be reported to stakeholders.

[□ Back to question](#)

## Answer 44

**Correct: (a).** `RidgeCV` picks the alpha from the supplied grid that maximizes cross-validated performance. `.alpha_` returns the chosen value; you can either refit a fresh `Ridge(alpha=ridge_cv.alpha_)` or use the `ridge_cv` object directly to predict.

[□ Back to question](#)

## Answer 45

**Correct: (e).** scikit-learn's `cross_val_score` always treats higher as better; for loss functions like MSE, this is achieved by returning the negative. Negate again to recover the positive MSE; take the square root for RMSE. The pattern `(-scores) ** 0.5` is the standard idiom.

[□ Back to question](#)



## Answer 46

**Correct: (d).** A 46-point gap between train  $R^2$  (0.97) and test  $R^2$  (0.51) is the textbook signature of overfitting. Standard fixes: regularization with cross-validated alpha, drop weak / collinear features, gather more training data, or simplify the model.

[□ Back to question](#)

## Answer 47

**Correct: (b) Elastic Net.** It combines L1 (selection) and L2 (stability with correlated features). Pure Lasso would arbitrarily zero all but one in each correlated group; pure Ridge would not eliminate any feature. Elastic Net offers the middle ground that real datasets usually need.

[□ Back to question](#)

## Answer 48

**Correct: (d).** When several features are highly correlated, Lasso's optimum is not unique — it can land on any one of them. Small perturbations in the data flip which feature wins. Ridge or Elastic Net handle correlated features more stably by spreading coefficient mass across them.

[□ Back to question](#)

## Answer 49

**Correct: (e).** A log transform converts multiplicative relationships into additive ones, which is exactly what linear regression assumes. It often produces more linear relationships and stabilizes residual variance — which is why log-transformed targets are common for prices, incomes, and other right-skewed quantities.

[□ Back to question](#)

## Answer 50

**Correct: (a)  $\approx 0.83$ .** Adjusted  $R^2 = 1 - (1 - R^2) \cdot (n - 1) / (n - p - 1) = 1 - 0.15 \cdot 99 / 89 \approx 0.833$ . The penalty for 10 predictors on 100 observations pulls the headline  $R^2$  of 0.85 down to about 0.83.

[□ Back to question](#)

## Answer 51

**Correct: (c).** A Random Forest with hundreds of trees and complex non-linear interactions is not designed for individual-level explanations. Linear / Ridge / Lasso models with named features and a single decision tree are far more transparent and easier to defend in a regulated context (banking, healthcare, hiring).

[□ Back to question](#)

## Answer 52

**Correct: (c).** With many features and a strong prior that only ~50 truly matter, Lasso (or Elastic Net if features cluster) is the right tool. Wrap the scaler and the regularizer in a Pipeline, choose alpha by cross-validation, and report the surviving feature subset alongside the coefficients.

[□ Back to question](#)

## Answer 53

**Correct: (b).** Unstable coefficients with stable predictions is the textbook signature of multicollinearity. Compute VIF, drop or combine the most-correlated features, and / or move to Ridge — which spreads coefficient mass across correlated features and gives stable, well-conditioned estimates.

[□ Back to question](#)

## Answer 54

**Correct: (c).** Linear regression measures associations, not causation. Confounding (warm weather drives both variables here) is one of the most common sources of spurious causal claims. Causal inference requires either a randomized experiment or a quasi-experimental design with a credible identification strategy.

[□ Back to question](#)

## Answer 55

**Correct: (a).** Train and test  $R^2$  that are CLOSE is a sign the model generalizes well — the model is not overfitting.  $R^2$  of 0.60 is realistic for noisy real-world credit data, where many drivers of repayment are unobserved. If business KPIs justify it, invest in better features or a more flexible model; otherwise an honest 0.60  $R^2$  is preferable to a flashy but overfit number that collapses in production.

[□ Back to question](#)